

Recursive Voice-Optimization Loop — Replit Agent Prompt

An RSI (recursive self-improvement) loop that learns which blog-copy *voice* maximizes engagement, using daily Facebook metrics as the feedback signal

How to use: Paste into the Replit AI agent as a task. It extends the main build (Replit-Agent-Prompt-JobGenie-AEO-GEO.md), especially Step 13's recursive loop) and your existing **Loop 2** (Meta Graph API publishing to the Job Genie Facebook Page + Instagram) and corpus admin dashboard. It reuses the credibility guardrails (§1 of the main prompt) and the citable-stats bank. It builds the *system*; humans approve every publish.

Objective

Build a closed, self-improving loop inside the Job Genie app that:

1. Pulls **daily metrics from the Job Genie Facebook Professional Dashboard** via the Meta Graph API (reach, views, impressions, engagement, clicks, follows).
2. **Attributes** each metric back to the *voice, hook, and structure* the corresponding blog/social variant was written in.
3. **Learns** which voices win — for which topic and persona — and by how much.
4. **Generates** the next batch of blog-post copy in multiple compelling voices (including the **Sabri Suby direct-response** style), biased toward proven winners while still exploring new voices.
5. **Recursively improves itself:** on a fixed cadence it rewrites its own voice library — retiring losers, spawning new voices from winning patterns — and logs the change.

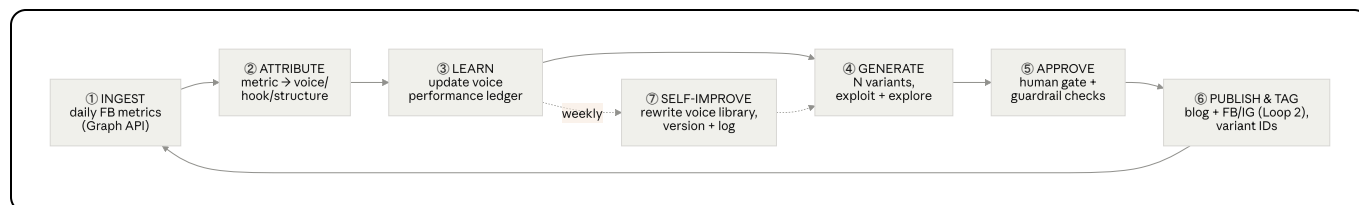
Everything routes through a **human approval gate** before publishing.

§0 — CRITICAL GUARDRAILS (enforce in everything you build)

1. **No autonomous publishing.** The loop drafts and ranks; a human approves in the corpus admin dashboard before anything posts to the blog, Facebook, or Instagram. Wire the gate so publishing is impossible without a recorded human approval.
2. **Meta Platform Terms compliance.** Use existing Loop 2 app credentials and the compliant human/agent split already in place. Only request metrics the app is authorized for. Never scrape the Professional Dashboard UI; use the Graph API insights edges. Respect rate limits and token scopes.

3. **API budget cap.** Hard cap LLM + API spend per loop run (default **\$2.00**, configurable in one place). Halt and alert on cap; never exceed silently.
 4. **Credibility guardrails carry over.** Every generated variant, in every voice, must pass the §1 checks from the main prompt: **no banned claims** (never the "70-80% hidden" myth or "AI auto-rejects in N seconds"), every statistic carries a named source, no fabricated trust signals. A voice being "compelling" never licenses a false claim — this applies with special force to the Sabri Suby voice's "big promise" (see voice spec).
 5. **Optimize for on-brand engagement, not vanity.** The objective function is a **composite** (reach × engagement quality × on-brand × downstream action), NOT raw views. Track "Get My Free Autopsy" clicks/conversions alongside views so the loop can't win by degrading into clickbait that spikes views and kills trust. If a variant maximizes views but tanks conversions or violates brand voice, it loses.
 6. **Never fabricate metrics.** All performance numbers come from the real Graph API or are marked "insufficient data." No invented engagement figures, ever.
 7. **Auditable mutation.** Every self-improvement change to the voice library is logged with a diff, the evidence that justified it, and a version bump. Nothing about the generation strategy changes without a changelog entry a human can review.
-

The loop (7 stations, three cadences)



Cadences: **daily** (ingest, attribute, learn) · **per-content-item** (generate → approve → publish) · **weekly** (self-improve the voice library).

Station ① — INGEST daily Facebook metrics (*daily*)

[AGENT BUILDS] A scheduled job (respecting the budget cap) that calls the Meta Graph API against the Job Genie Page (and linked Instagram Business account) and stores a daily time-series.

- Pull **page-level** and **per-post** insights. Target metric families: reach, impressions, video/post **views**, post engagements (reactions, comments, shares, saves), link/CTA clicks, profile visits, and follows/new page likes. For video, pull video-view metrics (e.g., 3-second and longer-watch views).

- **Resolve exact metric names against the current Graph API version at build time** — Meta renames/deprecates insight metrics frequently, so read the current insights reference rather than hard-coding a stale list. Use the existing Loop 2 Page access token and only the scopes already granted (`(read_insights)` etc.). If a needed permission isn't granted, surface it in the report rather than failing silently.
- Store to `(data/fb-metrics-daily.json)` (or the existing DB): one row per (post_id or page, metric, date, value). Idempotent — re-running a day overwrites, never double-counts.

Station ② — ATTRIBUTE metrics to voice & structure (*daily*)

[AGENT BUILDS] A join layer that maps each measured post back to the content variant that produced it, via the variant IDs stamped at publish time (Station ⑥).

- Each post resolves to its attributes: **voice** (Sabri Suby DR, empathetic peer, data analyst, contrarian myth-buster, storyteller — see voice set), **hook type**, **headline pattern**, **structure**, **topic/persona** (from the content pack clusters), post **time/day**, and **format** (image, carousel, video, text).
- **Normalize** to control for audience-size drift and volatility: compute engagement *rate* (per reach/impression), not raw counts. Require a **minimum impressions threshold** before a data point counts toward scoring (configurable) so tiny samples don't create false winners.
- Write `(data/variant-attribution.json)` linking variant_id → attributes → normalized metric time-series.

Station ③ — LEARN: update the voice performance ledger (*daily*)

[AGENT BUILDS] A scoring engine that maintains `(data/voice-ledger.json)`.

- Score each **voice × topic × persona × format** combination on the composite objective (§0.5). Read scores as **trends, volatility-adjusted** — a single viral post is noise; sustained lift is signal.
- Use an **explore/exploit** model (multi-armed bandit, e.g. Thompson sampling or UCB) so the ledger yields a *sampling policy*, not just a leaderboard: proven winners get more of the next batch, but a reserved share (default ~20–30%) always goes to exploration. This prevents premature convergence on a local maximum.
- Track uncertainty per arm; a voice with few impressions stays "under test," not "winning" or "losing," until it clears the sample threshold.

Station ④ — GENERATE copy variants (*per content item*)

[AGENT BUILDS] A generator that, for each new blog post/topic, produces **N variants**

(default 5, one per active voice) plus platform-specific social copy for each.

- The generator reads the sampling policy from Station ③: it **exploits** (weights toward voices/hooks/headline patterns winning for that topic+persona) while **exploring** (assigns some variants to under-tested or newly-spawned voices).
- Each variant is generated from the canonical Job Genie substance (content-pack FAQs, entity glossary, citable-stats bank) and stamped with a `variant_id` + full attribute tags.
- **Every variant runs the guardrail pre-check** (banned-claim scan, unsourced-stat scan, brand-voice check) *before* it reaches a human. Failing variants are auto-rejected with a reason, not surfaced.
- Respect the budget cap across all generations in a run.

Station ⑤ — APPROVE: human gate + guardrail checks (*per content item*)

[AGENT BUILDS] Extend the **corpus admin dashboard** with a variant-review queue.

- Show all N variants side by side with their voice label, predicted performance (from the ledger), and guardrail-check status (green/blocked).
- A human can approve, edit, or reject each variant. **Publishing is gated on a recorded approval** — no approval, no publish. Log approver + timestamp.
- Optionally let the human pick which variants go live as an A/B set vs. hold-outs.

Station ⑥ — PUBLISH & TAG (*per content item*)

[AGENT BUILDS] On approval, publish the blog variant to the site and distribute the matching social copy to Facebook/Instagram **through the existing Loop 2 Meta Graph API pipeline**.

- Stamp each published post with its `variant_id` so Station ② can close the loop. Store the mapping (post_id ↔ variant_id) in `data/published-variants.json`.
- Stagger publish times to avoid confounding voice effects with time-of-day effects where feasible, and record the publish timestamp as an attribute.

Station ⑦ — SELF-IMPROVE: rewrite the voice library (*weekly — the recursion*)

This is the recursive self-improvement core: the loop edits **the strategy it uses to generate copy**, not just the copy.

[AGENT BUILDS] A weekly job that proposes changes to `content/voice-library.json` and logs them to `docs/voice-library-changelog.md`:

- **Retire** voices (or hooks/headline patterns) that consistently lose across enough topics/impressions.
- **Spawn** new candidate voices by recombining winning patterns (e.g., "data analyst opening + Sabri-Suby CTA" if that combination outperforms either alone). New voices enter as exploration arms.
- **Re-weight** the base sampling policy toward what's winning per persona.
- **Refresh** each voice's spec/prompt with the concrete phrasings that are winning (and prune the ones that aren't).
- **Version bump + changelog entry:** date, what changed, the ledger evidence (which arms, what lift, over what sample), and the new version. **These proposals are themselves human-approvable** if they change generation behavior — respect §0.7. The loop improves itself, but under audit.

Then it feeds the updated library back into Station ④.

The voice test set (seed `content/voice-library.json`)

Five seed voices. Each must stay inside the credibility guardrails and Job Genie's honest-brand positioning. Specs are generation instructions the agent uses.

1. Sabri Suby — Direct Response (*the requested voice*) Channel the "Sell Like Crazy" DNA: open with a **bold, specific promise** paired with a **curiosity hook**; frame a clear **enemy** (the broken application black hole, ghost jobs, spray-and-pray culture); run **problem** → **agitate** → **solution**; reveal a "**hidden mechanism**" (the recruiter shortlist threshold / Recruiter-Fit Gap); use punchy short sentences, pattern interrupts, heavy second-person "you," a **value stack, risk reversal** (the free 100-Application Autopsy), and **one** strong CTA. **Hard constraint:** the big promise must be *literally true and sourced* — direct-response urgency, never hype or fabricated claims. No banned stats. This is the tension that makes it on-brand: maximally compelling *and* honest.

2. Empathetic peer ("been there") Warm, first-person, validating the specific frustration (the silence, the ghosting after a final round). Solidarity before advice. Conversational, low-hype. Leads with "you're not broken, the channel is."

3. Data-driven analyst Evidence-first. Lead with a sourced statistic from the citable-stats bank, then interpret it. Calm, credible, myth-busting with attribution. Wins trust-oriented audiences.

4. Contrarian myth-buster Pattern-interrupt built on a provocative-but-true reversal ("Stop tailoring your resume to the job post." "Auto-applying is sabotaging you."). Provocative framing, sourced substance. High scroll-stopping potential.

5. Story / narrative Open on a specific candidate scene (the 100-Application Autopsy narrative — one real-feeling person, one moment of silence), build an arc, land the lesson. Emotional resonance → shares.

The loop tests these against each other by topic/persona and evolves the set over time (Station ⑦).

Objective function & statistical rigor

- **Composite objective** (configurable weights), per §0.5: $\frac{w1 \cdot \text{reach_rate} + w2 \cdot \text{engagement_quality} + w3 \cdot \text{on_brand_score} - w4 \cdot \text{guardrail_risk} + w5 \cdot \text{CTA_action_rate}}{\text{}} \text{}$. Keep raw views visible but never the sole target.
 - **On-brand score**: a lightweight automated check (voice/claim alignment) plus optional human rating captured at approval.
 - **Volatility-adjusted trends** (per the methodology's I-8): score deltas vs. a rolling baseline, not single-post snapshots. FB reach re-rolls constantly; judge on trend.
 - **Minimum-sample gates** before any voice is called a winner or loser; keep uncertainty explicit; explore/exploit prevents overfitting to early noise.
 - **Attribution honesty**: where time-of-day, format, or topic confounds voice, note it; don't over-claim a voice effect that's really a format effect.
-

Data model / files the agent creates

- `data/fb-metrics-daily.json` — raw daily Graph API metrics (time-series).
- `data/variant-attribution.json` — `variant_id` → attributes → normalized metrics.
- `data/voice-ledger.json` — bandit scores + uncertainty per voice×topic×persona×format.
- `data/published-variants.json` — `post_id` ↔ `variant_id` mapping.
- `content/voice-library.json` — the evolving voice specs + sampling weights.
- `docs/voice-library-changelog.md` — the self-improvement audit log.
- `docs/recursive-voice-loop.md` — how the loop works, the human runbook, the budget-cap and approval-gate config.
- Corpus admin dashboard extensions: variant-review queue + a **voice-performance view** (which voices win, per persona, trending over time).

Deliverables & reporting

When built, run without publishing (dry run) and report: files created; the Graph API metrics successfully pulled (and any permissions still needed); the seed voice library; the objective-function config; the approval-gate wiring (prove publishing is blocked without approval); the budget-cap enforcement; and confirmation that **no metrics were fabricated and nothing published autonomously**. Provide the human runbook: how to review variants, read the voice-performance view, and approve weekly self-improvement proposals.

End of prompt. Build the loop; humans approve the publishes and the self-improvements.